

LCS

April 11, 2026

0.1 Longest Common Sequence - Implementazione e Testing

Si consideri il problema della **più lunga sottosequenza comune**, una *sottosequenza* di una sequenza s è una sequenza di *elementi* di s presi nello stesso **ordine**, ossia è individuata da una sequenza di *indici* di s , non necessariamente **adiacenti**.

Ad Esempio, considerate due stringhe $ABCD$ e $BCDA$ avremo come **LCS** la *sottosequenza* BCD .

Problema: Date due sequenze trovare la sottosequenza comune di lunghezza massima.

Iniziamo presentando l'algoritmo **Brute force** che, date le due sequenze s_1 , s_2 di lunghezza m e n rispettivamente, genera tutte le sottosequenze di s_1 , e per ognuna di esse controlla se questa è anche sottosequenza di s_2 , tenendo traccia della più lunga trovata. Ogni sottosequenza di s_1 corrisponde a un sottoinsieme degli indici della sequenza originale, poiché per ogni elemento la scelta è **binaria** (Mantengo o Scarto) ho un totale di 2^m sottosequenze generate. Per ognuna occorrono, nel caso peggiore, n passi per controllare se è anche sottosequenza di s_2 (Scorro tutta la sequenza alla ricerca di un match). Utilizzando una scansione simultanea della sottosequenza scelta e di s_2 , l'Algoritmo cerca il carattere corrente della sottosequenza all'interno di s_2 e, solo quando trova una corrispondenza, fa avanzare il puntatore al carattere successivo della prima, continuando a scorrere s_2 in avanti. La complessità è quindi $\Theta(n \times 2^m)$.

La formulazione induttiva della soluzione presentata è descritta nel modo seguente, dove $S_1[1...m]$ e $S_2[1...n]$ sono le due sequenze e $LCS(i, j)$ indica una sottosequenza comune di lunghezza massima tra i prefissi $S_1[1...i]$ e $S_2[1...j]$.

$$\begin{aligned} LCS(0, j) &= [], \forall j \in \{0, \dots, n\} \quad 0 \leq j \leq n \\ LCS(i, 0) &= [], \forall i \in \{0, \dots, m\} \quad 0 \leq i \leq m \end{aligned}$$

$\forall i, j \mid i \neq 0, j \neq 0$:

$$\begin{aligned} LCS(i, j) &= LCS(i-1, j-1) \cdot (i, j) \text{ se } S_1(i) = S_2(j) \\ LCS(i, j) &= \max(LCS(i-1, j), LCS(i, j-1)) \text{ altrimenti.} \end{aligned}$$

Informalmente, come **Base** della definizione si ha che la *LCS di due sequenze di cui una è vuota risulta vuota*. Il **Passo induttivo** corrisponde al caso in cui entrambi i prefissi sono non vuoti, si considerano i due casi seguenti:

- Se $S_1(i) = S_2(j)$ allora tutte le sottosequenze comuni sono tutte quelle precedenti e tutte quelle ottenute aggiungendo in fondo a una di queste la coppia (i, j) . e dunque $LCS(i, j) = LCS(i-1, j-1) \cdot (i, j)$.
- In caso contrario le sottosequenze comuni non potranno contenere la coppia (i, j) . Quindi $LCS(i, j)$ è la più lunga fra le due sequenze $LCS(i-1, j)$ e $LCS(i, j-1)$, questo è il caso

più complesso che va a generare un numero esponenziale di alberi di ricorsione necessari a valutare la sequenza più lunga fra tutte quelle possibili.

La **relazione di ricorrenza**, analizzati i due rami distinti di ricorsione (dovuti al calcolo di $\max(LCS(i-1, j), LCS(i, j-1))$) e il costo costante delle operazioni ad ogni passo, risulta essere, nel caso peggiore: $T(n, m) = T(n-1, m) + T(n, m-1) + 1$, come per le **Torri di Hanoi** questa equazione porta ad una *complessità esponenziale* pari a $\Theta(2^{n+m})$.
(Oltre ai file forniti, sono presenti due file di testing più piccoli).

```
[14]: # Imports
import IPython
import warnings

PATH = "../data"
```

```
[18]: # Text Files
F1 = "/stringA.txt"
F2 = "/stringB.txt"
#F1 = "/dna_sequence_100.txt"
#F2 = "/dna_sequence_200.txt"

# Opening and reading files
with open(PATH + F1, 'r') as f1:
    s1 = f1.read()
with open(PATH + F2, 'r') as f2:
    s2 = f2.read()
```

```
[ ]: # Alternatively using .py fibonacci sequences
import sys
sys.path.append(PATH)

from code_snippet_1 import fibonacci as fib1
from code_snippet_2 import fibonacci as fib2

N1 = 10
N2 = 12

s1_int = fib1(N1)
s2_int = fib2(N2)

s1 = [str(n) for n in s1_int]
s2 = [str(n) for n in s2_int]

#-----#
```

```
[ ]: # LCS - Brute Force Algorithm

def lcs_bf (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)
    return bf(s1, s2, m, n)

def bf(s1, s2, i, j):
    # Base
    if (i == 0 or j == 0):
        return [] # Empty List
    # Step
    # print("(" + s1[i-1] + ", " + s2[j-1] + ")")
    if (s1[i-1] == s2[j-1]):
        last = bf(s1, s2, i-1, j-1)
        # List concatenation
        return last + [(i,j)]
    else:
        lcs_a = bf(s1, s2, i-1, j)
        lcs_b = bf(s1, s2, i, j-1)
        # The longest List between the two
        return max(lcs_a, lcs_b, key=len)

# Output
res = lcs_bf (s1, s2)
lcs = ""
for i, j in res:
    lcs += s1[i-1]

print(f"LCS Indexes: {res}")
print(f"LCS Length: {len(res)}")
print(f"\nLCS: {lcs}")
```

LCS Indexes: [(2, 1), (3, 2), (4, 3)]

LCS Length: 3

LCS: BCD

Viene fornito un algoritmo migliore con la tecnica della **Programmazione dinamica**, in questo caso è semplice dimostrare l'applicabilità di tale approccio in quanto sono presenti sia una **Sottostruttura ottima** (la soluzione del problema contiene le soluzioni dei sottoproblemi necessari alla sua soluzione) che forme di **Sottoproblemi ripetuti** a causa dei quali avviene l'“*esplosione*” ricorsiva e che dunque possiamo memorizzare in strutture dati apposite.

Si costruisce una *matrice* **LCS** con $m + 1$ righe ed $n + 1$ colonne. In base alla definizione induttiva data sopra, la prima riga e la prima colonna vanno riempite con la sequenza vuota, di lunghezza 0. Si può poi procedere riga per riga (o colonna per colonna), l'ultima casella, cioè quella nell'angolo a destra in basso, conterrà la soluzione. In ogni casella non occorre memorizzare tutta la **LCS** ma

basta mettere:

- Se si ha lo stesso carattere sulla riga e sulla colonna, una “*freccia diagonale*”, aumentando di 1 la lunghezza rispetto alla casella puntata dalla freccia;
- Se sulla riga e sulla colonna ci sono due caratteri diversi, una freccia verso la cella di lunghezza maggiore fra la adiacente sopra e la adiacente a sinistra (se hanno uguale lunghezza viene scelto un criterio universale, ad esempio quella sopra). La lunghezza sarà la stessa di quella della cella puntata dalla freccia.

La **LCS** corrispondente a ciascuna casella si ottiene, seguendo le frecce a partire dall’ultima casella. Non è quindi necessario memorizzare i caratteri nelle celle, mentre la freccia è necessaria per ricostruire all’indietro la **LCS**, la lunghezza è fondamentale per calcolare le successive celle contigue.

```
[20]: import numpy as np
def lcs_dyn (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)
    L, R = dyn(s1, s2, m, n)

    print("Lengths Matrix:")
    print(L)
    print("References Matrix")
    print(R)

    res = ret_lcs(R, s1)
    print(f"\nLCS: {res}")
    print(f"LCS Length: {len(res)}")

# Function to create the two matrices, one for lengths and the other for
# references
def dyn(s1, s2, m, n):
    L = np.zeros((m+1, n+1), dtype=int)
    R = np.full((m+1, n+1), ' ', dtype=object)

    for i in range(1, m+1):
        for j in range(1, n+1):
            if (s1[i-1] == s2[j-1]):
                L[i,j] = L[i-1,j-1] + 1
                R[i,j] = ' '
            elif (L[i,j-1] > L[i-1,j]):
                L[i,j] = L[i,j-1]
                R[i,j] = '←'
            else:
                L[i,j] = L[i-1,j]
                R[i,j] = '↑'
    return L, R
```



```
[21]: import numpy as np
def lcs_dyn2 (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)

    res = dyn2(s1, s2, m, n)
    print(f"LCS Length: {res}")

def dyn2(s1, s2, m, n):
    # I can use two lists to remember the lengths that matter
    prev = np.zeros(n+1)
    cur = np.zeros(n+1)

    for i in range(1, m+1):
        for j in range(1, n+1):
            if (s1[i-1] == s2[j-1]):
                cur[j] = prev[j-1] + 1 # It simulates the diagonal arrow, the
↪upward left element + 1 ( )
            elif (cur[j-1] > prev[j]):
                cur[j] = cur[j-1] # It gets the previous element or the
↪one above (+, ↑)
            else:
                cur[j] = prev[j]
        prev = cur.copy() # It proceeds to the next line
    return prev[n] # Returns the last cell

# Output
lcs_dyn2(s1, s2)
```

LCS Length: 3.0

Considerati i tempi di esecuzione (ad esempio dei file dna_sequence_XXX.txt forniti), il primo algoritmo *ricorsivo* è notevolmente più lento, per i motivi sopra elencati, e questo dimostra il vantaggio assoluto della programmazione dinamica in questo tipo di problemi. Suppongo che un approccio **Greedy** a questo problema risulterebbe inefficiente in quanto la scelta *localmente ottima* **irrevocabile** non è quasi mai quella ideale, dunque creando la soluzione incrementalmente si presenterebbero soluzioni subottimali.

Baldini Filippo - 6393212